

CS 218

Programmable & Interoperable Blockchain

PROJECT ASSIGNMENT

10 Projects · Teams of 4 · 40 Marks Each

Toolchain: Hardhat / Foundry · Language: Solidity · Library: OpenZeppelin

Presentation Dates	25 April, 26 April 2026
Submission Deadline	30 April 2026, 11:30 PM
Submission Method	Share your public GitHub repository link

What to Submit

Create a public GitHub repository with a meaningful name (e.g., cs218-crowdfunding-team5).

Warning: Private repositories will incur a -2 mark penalty. The teaching team must be able to clone and run your tests without any access request.

Repository must contain:

- All Solidity source files (.sol)
- Hardhat or Foundry configuration files (hardhat.config.js or foundry.toml)
- Test files covering all major functions
- A deployment script (scripts/deploy.js or script/Deploy.s.sol)
- README.md — setup instructions, team member names and roll numbers
- report.pdf — project report (see below)

Report must include:

- Problem statement and why blockchain is the right tool here
- Architecture diagram: which functions exist, how they relate
- Security analysis: which attacks are prevented and how
- Gas optimisation: gas report output + at least one before/after comparison
- Test coverage report (forge coverage or hardhat-coverage output)

Toolchain Guide

Option A — Hardhat (Recommended, Node.js / TypeScript)

Hardhat is the most widely adopted Ethereum development environment. It is JavaScript/TypeScript-first and includes `console.log` inside Solidity contracts for debugging.

Setup:

```
npm init -y

npm install --save-dev hardhat @nomicfoundation/hardhat-toolbox

npm install @openzeppelin/contracts

npx hardhat init    # choose TypeScript project
```

Daily workflow:

```
npx hardhat compile           # compile all .sol files

npx hardhat test              # run tests + show gas table

npx hardhat node              # start local blockchain

npx hardhat run scripts/deploy.ts --network localhost  # deploy
```

Useful plugins (included in hardhat-toolbox):

- hardhat-gas-reporter — automatic gas table after tests
- hardhat-coverage — test coverage report

Time travel in tests (essential for deadline-based contracts):

```
const { time } = require("@nomicfoundation/hardhat-network-helpers");
await time.increase(7 * 24 * 60 * 60);    // advance 7 days
await time.increaseTo(deadline + 1);      // jump to a specific timestamp
```

Option B — Foundry (Advanced, Rust-based, Tests in Solidity)

Foundry is the tool of choice for security researchers. Tests are written in Solidity, forcing you to think in the same language as the contracts. It is dramatically faster for large test suites.

Setup:

```
curl -L https://foundry.paradigm.xyz | bash

foundryup

forge init my-project && cd my-project

forge install OpenZeppelin/openzeppelin-contracts
```

Daily workflow:

```
forge build                # compile

forge test                 # run all tests

forge test --gas-report    # run tests + gas table

forge test -vvvv          # full stack traces for debugging

forge coverage             # test coverage report

forge script script/Deploy.s.sol --rpc-url http://localhost:8545 --broadcast
```

Time travel in Foundry tests (cheatcodes):

```
vm.warp(block.timestamp + 7 days);    // advance time

vm.prank(alice);                     // impersonate an address

vm.expectRevert();                   // assert next call reverts

vm.deal(alice, 10 ether);             // fund an address
```

OpenZeppelin — Required for All Projects

OpenZeppelin is the industry-standard audited smart contract library. Using it is mandatory — do not write your own Ownable, ReentrancyGuard, or token implementations from scratch. Every such attempt introduces vulnerabilities that have already been solved.

Contract / Module	Purpose	Used in Projects
Ownable	Adds onlyOwner modifier	All projects
ReentrancyGuard	Prevents reentrancy attacks on ETH-sending functions	1,2,4,5,7,8,9,10
ERC20	Standard fungible token	9
ERC721 & ERC721URIStorage	Standard NFT	7
AccessControl	Multi-role permissions	3,6
IERC2981	Standard royalty interface for NFT resales	7
SafeERC20	Safe wrappers for ERC-20 transfers	9

Marks Breakdown — 40 Marks (Applies to All 10 Projects)

Category	Max Marks	Sub-criteria	Marks
Smart Contract Correctness	14	Core logic correct (all required functions implemented and behave correctly)	8
		Access control (only authorised addresses can call privileged functions)	3
		Edge cases handled (zero values, empty arrays, expired deadlines, etc.)	3
Security	6	Reentrancy guard on all ETH-sending functions	2
		Input validation (all require() statements; no over/underflow)	2
		Visibility correct (public/private/internal/external assigned well)	2
OpenZeppelin Usage	2	OZ contracts used where applicable (Ownable, ReentrancyGuard, token standards)	2
Gas Optimisation	5	Gas report submitted (from hardhat-gas-reporter or forge --gas-report)	2
		One concrete optimisation shown with before/after numbers and explanation	3
Testing	7	Happy-path tests for every core function	3
		Revert/failure tests (unauthorised or invalid calls revert correctly)	2
		Coverage report showing 70% or greater line coverage	2
DApp Frontend	4	Wallet connection (MetaMask, contract ABI loaded, balance displayed)	2
		At least one state-changing function callable from UI with status feedback	2
Documentation & Code	2	README complete (setup steps, team members, roll numbers)	1
		NatSpec comments (@notice, @param) on all public functions	1
TOTAL	40		

Deductions (applied on top of 40 marks)

-2 marks	GitHub repository is private at submission
-3 marks	Contract does not compile

-1 mark per	Missing team member name in README
-2 marks	No tests are submitted

General Workflow for All Projects

1. **Design** — Draw the state diagram on paper. List every function, its inputs, outputs, who can call it, and what state it changes. Do not open an editor yet.
2. **Write** — Implement functions one at a time. Start with structs and state variables, then simple view functions, then complex state-changing functions. Add OpenZeppelin imports from the start.
3. **Compile** — Run `npx hardhat compile` or `forge build`. Fix all warnings — in Solidity, warnings are often latent bugs.
4. **Test** — Write one test per function (happy path first, then revert tests). Aim for >70% line coverage.
5. **Gas Report** — Run `npx hardhat test` (with gas reporter) or `forge test --gas-report`. Find the most expensive function, make one targeted optimisation, and record before/after numbers.
6. **Frontend** — Build a minimal React app with `ethers.js` or `wagmi`. Connect MetaMask, display contract state, and wire at least one write transaction.
7. **Deploy** — Deploy to a local node first (`npx hardhat node` or `anvil`). Confirm everything works.
8. **Report** — Write the PDF report following the structure in "What to Submit".

On-chain vs off-chain — the cardinal rule: Only store data that requires immutability, transparency, or programmatic verification on-chain. Everything else (images, documents, user profiles) belongs off-chain (IPFS, Arweave, or a server). Storing a keccak256 hash on-chain and the actual data off-chain gives you verifiability without the cost.

Project Specifications

Project 1 — Decentralised Book Rental

Why This Project? *Escrow patterns are foundational to almost every DeFi protocol. This project teaches you how to hold ETH, release it conditionally, and handle disputes — all in one clean system.*

A peer-to-peer platform where owners list physical items for rent, renters pay a deposit plus rental fee held in the contract, and the deposit is refunded or withheld based on the condition reported at return. A lightweight on-chain dispute mechanism prevents either party from

blocking resolution unilaterally.

Required Contract Functions

- `listItem(string name, uint pricePerDay, uint depositWei)` — owner registers an item; emits `ItemListed`
- `rentItem(uint itemId)` payable — renter sends deposit + first-day fee; status → `Rented`
- `returnItem(uint itemId)` — renter marks item returned; owner must confirm within 48 hours
- `confirmReturn(uint itemId)` — owner confirms good condition; deposit refunded to renter
- `raiseDispute(uint itemId)` — either party escalates; funds frozen; emits `DisputeRaised`
- `resolveDispute(uint itemId, bool toRenter)` — arbitrator (set at deployment) releases funds
- `getItem(uint itemId) view` — returns name, owner, renter, status, deposit held

Implementation Notes & Tooling

- Status enum: `Available` | `Rented` | `AwaitingConfirm` | `InDispute` | `Closed`
- Struct: `Item { name, owner, renter, pricePerDay, deposit, rentedAt, status }`
- `OpenZeppelin`: `Ownable` for item listing control; `ReentrancyGuard` on `rentItem` and `resolveDispute`
- Use `block.timestamp` to enforce the 48-hour confirmation window; after expiry anyone can trigger auto-refund
- Store deposited ETH in the contract (never forward it to owner before return is confirmed)
- Hardhat: `await time.increase(48 * 3600)` to test the auto-refund timeout

Testing Requirements

- Non-owner cannot list an item
- Renting an already-rented item reverts
- Happy path: `list` → `rent` → `confirm return` → `deposit refunded`
- Dispute path: `renter raises dispute` → `arbitrator resolves in owner's favour` → `deposit goes to owner`
- 48-hour timeout: `advance time`, `call auto-refund`, `verify renter gets deposit`

Instructor Tip

Teams that implement the dispute resolution and write a test that shows the dispute path from start to finish will almost certainly achieve full marks in correctness and security. The dispute mechanism is what separates this project from a toy escrow.

Project 2 — Decentralised Freelance Platform

Why This Project? *Escrow-based payment protocols are the basis of decentralised commerce. Learning how to hold funds between two parties, enforce time limits, and maintain a reputation system without a central server is a transferable skill that applies to supply chain, insurance, and any multi-party agreement.*

Freelancers post services with a fixed price. Clients hire a freelancer and the fee is escrowed in the contract. On job completion (confirmed by the client), funds release to the freelancer. If neither party acts within a deadline, either can cancel and the client is refunded. A simple on-chain reputation score is maintained per freelancer.

Required Contract Functions

- `offerService(string title, string description, uint priceWei)` — freelancer lists a service
- `hireFreelancer(uint serviceId)` payable — client sends exact price; funds escrowed; emits `JobCreated`

- `confirmCompletion(uint jobId)` — client confirms; ETH released to freelancer; emits `JobCompleted`
- `cancelJob(uint jobId)` — either party cancels before work is accepted; client refunded
- `rateFreelancer(uint jobId, uint8 score)` — client rates 1-5 after completion; score stored on-chain
- `getReputation(address freelancer)` view — returns $(avgScore * 100, totalJobs)$ to avoid decimals
- `getService(uint servId)` view / `getJob(uint jobId)` view

Implementation Notes & Tooling

- Status enums: Service: Listed | Hired | Completed | Cancelled; Job: Active | Done | Cancelled
- Auto-cancel timeout: 7 days after hiring; use `block.timestamp` comparison in `cancelJob`
- OpenZeppelin: `ReentrancyGuard` on `hireFreelancer` and `confirmCompletion`
- Hardhat: test timeout with `await time.increase(8 * 24 * 3600)`
- Freelancer cannot withdraw independently — funds only move via `confirmCompletion` or `cancelJob`

Testing Requirements

- Only the client can confirm completion
- Freelancer cannot self-confirm or self-release funds
- Rating a freelancer twice from the same job reverts
- 7-day auto-cancel: advance time, cancel, verify refund
- Reputation: hire same freelancer twice with scores 4 and 2; average should be 3

Instructor Tip

The reputation system is the distinguishing feature here. A team that stores ratings correctly and returns a sensible average will demonstrate they understand mappings and integer arithmetic on-chain.

Project 3 — DAO Governance Contract

Why This Project? *DAO governance is how all major DeFi protocols make decisions about hundreds of millions of dollars. Every professional Ethereum developer is expected to understand governance contracts, token-weighted voting, quorum, and on-chain execution.*

A governance system used by DAOs. Members hold a governance token (ERC-20) that gives voting power. Any member can create a proposal. After a voting period, if quorum is reached and majority votes yes, the proposal can be executed — triggering a real on-chain call.

Required Contract Functions

- `constructor(address governanceToken)` — sets the ERC-20 token used for voting power
- `createProposal(string description, address target, bytes calldata data, uint votingPeriod)` — any token holder can create
- `vote(uint proposalId, bool support)` — one address, one vote per proposal; weight = token balance at proposal creation
- `executeProposal(uint proposalId)` — callable after deadline if quorum met and majority voted yes
- `cancelProposal(uint proposalId)` — creator can cancel before voting starts
- `getProposal(uint proposalId)` view — status, yesVotes, noVotes, quorumReached, deadline, executed
- Deploy a simple ERC-20 governance token contract (`MintableToken`) and distribute to test accounts

Implementation Notes & Tooling

- OpenZeppelin: ERC20 for the governance token; AccessControl for admin roles (e.g., admin can update quorum percent)
- Quorum: at least 30% of total token supply must have voted (yes + no combined) for the proposal to be executable
- Snapshot: record each voter's token balance at the time of createProposal (not at time of vote) to prevent flash-loan attacks
- Status enum: Pending | Active | Defeated | Succeeded | Executed | Cancelled
- Foundry is strongly recommended: use vm.warp() for voting period, vm.prank() to switch voter identities
- The on-chain execution target in tests can be a simple Treasury contract with a release(address, uint) function

Testing Requirements

- Non-token-holder cannot vote
- Double-voting on the same proposal reverts
- A proposal below quorum cannot execute even if all votes are yes
- Time: proposal is votable during window; executeProposal reverts before deadline, succeeds after
- After a successful vote, executeProposal actually calls the target contract and changes its state
- A defeated proposal cannot be executed

Instructor Tip

The execution test — where you show a successful governance vote actually triggers a state change in a Treasury contract — separates a strong submission from an average one. It demonstrates you understand contract-to-contract calls, the core of DeFi composability.

Project 4 — Lottery System with Commit-Reveal Randomness

Why This Project? *On-chain randomness is one of the hardest problems in blockchain. The commit-reveal scheme is the simplest correct solution that requires no external oracle. Understanding why it works is fundamental to building fair on-chain games, NFT mints, and any contract with randomness.* A decentralised lottery where participants buy tickets during an open phase. When the sale closes, the owner commits to a secret random value (hashed). After the commitment is publicly visible, the owner reveals the secret. The contract verifies the hash and uses the secret to select a winner deterministically. The winner claims the prize pool.

Required Contract Functions

- buyTicket() payable — adds msg.sender to participants; emits TicketPurchased
- closeSale() — owner closes ticket sales; phase → Committed
- commitHash(bytes32 hash) — owner submits keccak256(abi.encodePacked(secret)) before revealing
- revealAndDraw(bytes32 secret) — owner reveals; contract verifies hash; computes winner index
- claimPrize() — winner withdraws the prize pool; emits PrizeClaimed
- getLotteryInfo() view — phase, ticketPrice, numParticipants, prizePool, winner (if drawn)

Implementation Notes & Tooling

- Phases: Open → SaleClosed → Committed → Drawn
- Commit-reveal: Owner runs keccak256(secret) off-chain and submits the hash first. When revealed, if the hash does not match, the transaction reverts.

- Winner formula: `uint winner = uint(keccak256(abi.encodePacked(secret, block.number))) % participants.length;`
- Why NOT `block.timestamp` alone: a miner who buys a ticket can choose which timestamp to use when mining the draw block, biasing the result. Document this in your report.
- OpenZeppelin: `ReentrancyGuard` on `claimPrize`; `Ownable` for phase transitions

Testing Requirements

- Ticket purchases revert after sale is closed
- Revealing a wrong secret (hash mismatch) reverts
- Only the winner can claim the prize
- Prize pool exactly equals the sum of all ticket payments
- A second call to `revealAndDraw` after winner is set reverts
- The contract cannot be drained: a non-winner calling `claimPrize` reverts

Instructor Tip

The security analysis in the report is as important as the code. You must explain why `keccak256(block.timestamp)` is exploitable, how `commit-reveal` fixes it, and what residual trust in the owner remains (the owner knows the secret). Teams that address this residual trust honestly will stand out.

Project 5 — Decentralised Auction System

Why This Project? *The auction contract is a classic teaching vehicle because it combines time-based logic, ETH escrow, the withdrawal pattern (the single most important reentrancy protection pattern), and event-driven UIs. Every concept directly maps to real-world DeFi.*

Any user can create an English (open ascending-bid) auction for a named item with a minimum starting price and a duration in seconds. Bidders send ETH; the highest bid wins. When the auction expires, the seller can end it to receive payment, and all losing bidders can withdraw their bids.

Required Contract Functions

- `createAuction(string itemName, uint startingPrice, uint durationSeconds)` returns `(uint auctionId)`
- `placeBid(uint auctionId)` payable — must exceed current highest bid; previous high bid made refundable
- `endAuction(uint auctionId)` — callable by anyone after deadline; transfers ETH to seller; emits `AuctionEnded`
- `withdrawBid(uint auctionId)` — losing bidder reclaims ETH (withdrawal pattern — do NOT push ETH)
- `getAuction(uint auctionId)` view — `itemName`, `seller`, `highestBid`, `highestBidder`, `deadline`, `ended`

Implementation Notes & Tooling

- Withdrawal pattern is mandatory: store refundable amounts in `mapping(uint => mapping(address => uint)) pendingReturns` and let bidders pull their own ETH. Never push ETH inside `placeBid`.
- OpenZeppelin: `ReentrancyGuard` on `withdrawBid` (classic reentrancy target); `Ownable` not needed — auctions are permissionless
- Note in your report: `block.timestamp` can be manipulated by validators by ~15 seconds. Discuss whether this matters for your auction.
- Hardhat time-travel: `await time.increaseTo(deadline + 1)`
- Advanced (sealed bid): bidders commit `keccak256(amount, nonce)`, reveal after deadline, highest revealed bid wins

Testing Requirements

- Bids below the current highest bid revert
- endAuction reverts before the deadline
- Losing bidders successfully withdraw their exact bid amount
- Reentrancy test: deploy a malicious contract that re-enters withdrawBid; verify ReentrancyGuard prevents the drain
- After endAuction, no further bids are accepted
- The seller receives exactly highestBid (use $\geq$ comparison due to gas)

Instructor Tip

The reentrancy test with a malicious contract is the most impressive deliverable. Writing the attacker contract in your test suite shows you genuinely understand the vulnerability, not just that you added a guard.

Project 6 — Decentralised Identity Verification

Why This Project? *Self-sovereign identity (SSI) is one of the most practically important blockchain use cases. Employers, universities, and governments are actively deploying identity systems on Ethereum. Understanding how identity hashes, on-chain credentials, and contract composability work is directly employable knowledge.*

Users register a hash of their identity document on-chain. Authorised verifiers approve or revoke verification status. Other smart contracts can call `isVerified()` to gate access — a verified-only auction, a KYC-gated token sale, etc. This demonstrates contract composability.

Required Contract Functions

- `registerIdentity(bytes32 identityHash)` — user commits hash of their document; status → Pending
- `verifyIdentity(address user)` — authorised verifier approves; status → Verified
- `revokeIdentity(address user)` — verifier revokes; emits `IdentityRevoked`
- `addVerifier(address)` / `removeVerifier(address)` — admin (owner) manages verifier list
- `isVerified(address user)` view returns (bool) — callable by other contracts
- `getIdentityRecord(address user)` view — returns hash, status, verifiedBy, timestamp
- Deploy a second `KYCGatedAuction` contract that restricts bidding to verified addresses only

Implementation Notes & Tooling

- OpenZeppelin: Ownable for verifier management; `AccessControl` is preferred if you want multiple roles (`ADMIN_ROLE`, `VERIFIER_ROLE`)
- Status enum: `NotRegistered` | `Pending` | `Verified` | `Revoked`
- Off-chain privacy: the actual document never goes on-chain — only its `keccak256` hash. Explain in your report why this matters under GDPR.
- Composability demo: `KYCGatedAuction` should call `IdentityVerifier.isVerified(msg.sender)` inside `placeBid` and revert with 'KYC required' if false

Testing Requirements

- A non-verifier cannot approve identities
- An unregistered user is not verified
- Revoking a verified identity correctly changes status to `Revoked`
- Composability: deploy `KYCGatedAuction`, verify that an unverified address's bid reverts
- After identity verification, the same address can bid successfully

Instructor Tip

The composability test — two contracts interacting — is what makes this project unique. Many teams complete the identity contract correctly but skip the KYCGatedAuction. That second contract is where the real learning happens.

Project 7 — NFT Event Ticketing (ERC-721)

Why This Project? *NFTs are the standard way to represent unique, transferable ownership on-chain: concert tickets, game items, real-estate deeds, software licenses. ERC-721 plus ERC-2981 royalties is the production standard used by OpenSea and Blur.*

An event organiser deploys the contract, creates events (name, max tickets, price, royalty percent), and mints ERC-721 NFT tickets to buyers. Ticket holders can list tickets for secondary sale on the contract's internal marketplace. Each resale pays the original organiser a royalty, implemented to the ERC-2981 standard.

Required Contract Functions

- `createEvent(string name, uint maxTickets, uint priceWei, uint96 royaltyBps)` — organiser creates; emits `EventCreated`
- `buyTicket(uint eventId)` payable — mints ERC-721 token to buyer; transfers ETH to organiser; emits `TicketMinted`
- `listForResale(uint tokenId, uint priceWei)` — holder puts ticket on internal marketplace
- `buyResaleTicket(uint tokenId)` payable — buyer pays; seller receives price minus royalty; organiser receives royalty
- `cancelResaleListing(uint tokenId)` — holder removes listing
- `tokenURI(uint tokenId)` view — returns JSON metadata URI (IPFS link or mock string)
- `royaltyInfo(uint tokenId, uint salePrice)` view — EIP-2981 implementation; returns (receiver, royaltyAmount)
- `getEvent(uint eventId)` view / `getResaleListing(uint tokenId)` view

Implementation Notes & Tooling

- OpenZeppelin base: `ERC721URIStorage` (inherits ERC-721, adds `tokenURI` per token); also implement `IERC2981` for royalties
- Override `supportsInterface()` to return true for both ERC-721 and ERC-2981 interface IDs
- Royalty in basis points: 500 bps = 5%. $\text{royaltyAmount} = \text{salePrice} * \text{royaltyBps} / 10000$
- Use a counter for `tokenId` (OZ Counters or a simple `uint`); increment before minting
- Internal marketplace: `mapping(uint => ResaleListing)` where `listing = { seller, price, active }`
- ReentrancyGuard: apply to `buyTicket` and `buyResaleTicket` as they transfer ETH
- Frontend tip: use `ethers.js` to call `tokenURI`, parse the JSON, and display the ticket image and event name

Testing Requirements

- Buying beyond `maxTickets` reverts
- A non-owner cannot list a ticket for resale
- Royalty split: on a resale of 1 ETH with 10% royalty, verify seller gets 0.9 ETH and organiser gets 0.1 ETH
- ERC-721 `ownerOf()` updates correctly after each transfer
- EIP-2981: call `royaltyInfo(tokenId, 1 ether)` and verify returned amounts
- A cancelled listing cannot be purchased

Instructor Tip

The royalty split test is where most teams lose marks — they implement `royaltyInfo` correctly but forget to actually deduct the royalty in `buyResaleTicket`. Make sure the ETH flow in the resale function matches what `royaltyInfo` promises.

Project 8 — Decentralised Crowdfunding

Why This Project? *Crowdfunding is the cleanest possible demonstration of conditional escrow — the two-outcome structure (success/failure) maps directly onto smart contract logic. The refund pattern it requires is exactly the same withdrawal pattern used in DEXes, lending protocols, and staking contracts to safely return ETH to users.*

Campaign creators set a funding goal (in wei) and a deadline (timestamp). Contributors send ETH during the active window. If the goal is met by the deadline, the creator can withdraw all funds. If the deadline passes without reaching the goal, every contributor can individually reclaim their exact contribution.

Required Contract Functions

- `createCampaign(uint goalWei, uint deadlineTimestamp)` returns (uint campaignId)
- `contribute(uint campaignId)` payable — adds `msg.sender`'s contribution; emits `Contributed`
- `withdrawFunds(uint campaignId)` — creator calls after deadline if goal met; emits `FundsWithdrawn`
- `refund(uint campaignId)` — contributor calls if deadline passed and goal not met; emits `Refunded`
- `getCampaign(uint campaignId)` view — goal, raised, deadline, creator, status
- `getContribution(uint campaignId, address contributor)` view — how much a specific address contributed
- Advanced: `createMilestone(uint campaignId, string desc, uint amount)` / `voteMilestone` / `releaseMilestone`

Implementation Notes & Tooling

- Contributions: `mapping(uint => mapping(address => uint))` contributions
- OpenZeppelin: `ReentrancyGuard` on `withdrawFunds` and `refund` (both send ETH)
- Status enum: `Active | Successful | Failed | FundsWithdrawn`
- Validate: goal must be > 0 ; deadline must be in the future (`block.timestamp + minDuration`)
- Hardhat: `await time.increaseTo(deadline + 1)` to simulate campaign expiry
- Milestone extension: creator requests tranche N; contributors with > 0 balance each get one vote; if majority approve, that tranche is released

Testing Requirements

- Successful campaign: goal reached before deadline, creator withdraws, contributors cannot refund
- Failed campaign: deadline passes, goal not met, each contributor gets their exact contribution back
- Partial: 3 contributors reach the goal together; all three cannot refund after creator withdraws
- Creator cannot withdraw before the deadline (even if goal is met)
- Contributing after the deadline reverts
- Zero-contribution refund attempt reverts

Instructor Tip

The time-travel tests (advance time, then verify state transitions) are what most students find surprising — the fact that Hardhat and Foundry let you move the blockchain clock is one of the most powerful testing techniques in smart contract development. Master it here and you will use it in every future project.

Project 9 — ERC-20 Token with Staking Rewards

Why This Project? *ERC-20 is the most important standard in all of DeFi — every stablecoin, governance token, and yield-bearing asset is an ERC-20. Staking is the simplest DeFi primitive: it teaches time-weighted balances, reward accrual, and the relationship between a token contract and a protocol contract.*

Deploy a fungible ERC-20 token (your own currency). Then deploy a separate Staking contract that lets holders lock their tokens for a chosen duration and earn additional tokens as rewards. The reward rate is proportional to the staked amount and duration. Unstaking returns the principal; claiming distributes the earned reward. The two contracts are linked: the staking contract mints reward tokens.

Required Contract Functions

- CS218Token (ERC-20): name, symbol, decimals; mint() restricted to Staking contract and owner
- stake(uint amount) — user approves then stakes; records (amount, startTimestamp); emits Staked
- unstake(uint amount) — user withdraws staked principal; remaining stake continues accruing
- claimRewards() — computes and mints accrued rewards to user; resets accrual timer
- getPendingRewards(address user) view — returns accrued rewards without claiming
- getStakedBalance(address user) view — returns currently staked amount
- setRewardRate(uint tokensPerDayPer1000Staked) — owner updates rate; emits RateUpdated

Implementation Notes & Tooling

- Deploy two contracts: CS218Token.sol and StakingRewards.sol. Pass token address to Staking constructor.
- Reward formula: $\text{rewards} = (\text{staked} * \text{rate} * (\text{block.timestamp} - \text{lastClaimTime})) / (1000 * 86400)$
- Use token.approve(stakingAddress, amount) before calling stake(); show this in your test setup and README
- OpenZeppelin: ERC20 for token; Ownable on both contracts; ReentrancyGuard on unstake and claimRewards
- Hardhat time-travel: stake, advance 7 days, claimRewards, verify token balance increased by expected amount
- Edge case: user who stakes then immediately unstakes should earn 0 rewards (or dust — document the rounding behaviour)

Testing Requirements

- Staking transfers tokens from user to contract (check both balances)
- Rewards after 1 day are half the rewards after 2 days (linear accrual)
- A user who stakes 2x as many tokens earns 2x the rewards in the same period
- claimRewards resets the accrual timer (second immediate claim returns 0 or dust)
- Unstaking returns exactly the staked amount (no rewards in principal)
- Staking 0 tokens reverts
- A non-owner cannot call setRewardRate

Instructor Tip

The double-check test — stake X, advance T days, claim, check balance — is the core of this project. Students who get the reward formula right but forget to account for token decimals (18 decimal places) will see numbers 10^{18} times too large. Write the formula on paper first, then code it.

Project 10 — Multi-Signature Wallet

Why This Project? *Multi-sig wallets are the standard security practice for any serious on-chain fund management — every DAO treasury, every protocol's admin key, every exchange's hot wallet uses multi-sig. Understanding how M-of-N approval works, how to prevent replay attacks with nonces, and how to execute arbitrary calldata is advanced but essential Solidity knowledge.*

A shared treasury wallet that requires M-of-N owner signatures before any transaction executes. Any owner can submit a transaction (recipient, value, calldata). Each owner independently approves or revokes approval. Once M approvals are collected, any owner can execute the transaction. This is the exact architecture of Gnosis Safe, the most widely used multi-sig in DeFi, holding over \$50 billion in assets.

Required Contract Functions

- `constructor(address[] owners, uint requiredApprovals)` — sets owner list and M threshold
- `submitTransaction(address to, uint value, bytes calldata data)` returns (uint txId) — any owner proposes
- `approveTransaction(uint txId)` — owner signals approval; emits Approved
- `revokeApproval(uint txId)` — owner withdraws their approval before execution
- `executeTransaction(uint txId)` — callable by any owner once M approvals reached; executes the call; emits Executed
- `getTransaction(uint txId)` view — to, value, data, approvalCount, executed
- `getApprovers(uint txId)` view — list of addresses that have approved
- `receive()` external payable — allows wallet to receive ETH

Implementation Notes & Tooling

- No OpenZeppelin base needed — this is a from-scratch contract. But use ReentrancyGuard on `executeTransaction`
- Duplicate owner prevention: check for duplicate addresses in the constructor; revert if any address is `address(0)` or appears twice
- Execution: use `(bool success,) = to.call{value: value}(data);` and `require(success)`
- Mark transaction as executed BEFORE the external call (checks-effects-interactions pattern)
- Foundry is ideal: use `vm.prank()` to simulate each owner's approval independently in a single test function
- Frontend: show pending transactions, how many approvals each has, and a button for each owner to approve or execute

Testing Requirements

- A non-owner cannot submit or approve transactions
- A transaction with fewer than M approvals cannot be executed
- Full happy path: submit → M owners approve → execute → verify state change in target contract
- Revoking an approval decrements the count and prevents execution until another approval is given
- An already-executed transaction cannot be executed again

- ETH receipt: send ETH to the wallet, then execute a transaction that sends it out
- Duplicate owners in constructor reverts

Instructor Tip

The calldata execution test — where you submit a transaction with encoded function calldata and verify it executes correctly on a target contract — demonstrates the most advanced skill in the course. Write a simple Counter contract, encode an increment call as bytes, submit it through the multi-sig, execute it, and verify the counter incremented.

Resources & References

Official Documentation

- Solidity: <https://docs.soliditylang.org>
- Hardhat: <https://hardhat.org/docs>
- Foundry Book: <https://book.getfoundry.sh>
- OpenZeppelin Contracts: <https://docs.openzeppelin.com/contracts>
- Ethereum Yellow Paper: <https://ethereum.github.io/yellowpaper/paper.pdf>

Development Tools

- Remix IDE: <https://remix.ethereum.org>
- MetaMask: <https://metamask.io>

Learning Resources

- CryptoZombies: <https://cryptozombies.io>
- Alchemy University: <https://university.alchemy.com>
- Patrick Collins — Foundry Full Course (YouTube): search 'Foundry Full Course 2024'
- Smart Contract Best Practices: <https://consensys.github.io/smart-contract-best-practices>
- Awesome Ethereum: <https://github.com/ttumiel/Awesome-Ethereum>